

AI Engineering

Building Applications with Foundation Models

Chip Huyen

O'REILLY®

[OceanofPDF.com](https://oceanofpdf.com)

In the early days of foundation models, when the strongest models were commercial with limited finetuning access, there weren't many competitive models available for finetuning. However, as the open source community proliferates with high-quality models of all sizes, tailored for a wide variety of domains, finetuning has become a lot more viable and attractive.

Reasons Not to Finetune

While finetuning can improve a model in many ways, many of these improvements can also be achieved, to a certain extent, without finetuning. Finetuning can improve a model's performance, but so do carefully crafted prompts and context. Finetuning can help with structured outputs, but many other techniques, as discussed in [Chapter 2](#), can also do that.

First, while finetuning a model for a specific task can improve its performance for that task, it can degrade its performance for other tasks.¹ This can be frustrating when you intend this model for an application that expects diverse prompts.

Imagine you need a model for three types of queries: product recommendations, changing orders, and general feedback. Originally, the model works well for product recommendations and general feedback but poorly for changing orders. To fix this, you finetune the model on a dataset of (query, response) pairs about changing orders. The finetuned model

might indeed perform better for this type of query, but worse for the two other tasks.

What do you do in this situation? You can finetune the model on all the queries you care about, not just changing orders. If you can't seem to get a model to perform well on all your tasks, consider using separate models for different tasks. If you wish to combine these separate models into one to make serving them easier, you can also consider merging them together, as discussed later in this chapter.

If you're just starting to experiment with a project, finetuning is rarely the first thing you should attempt. Finetuning requires high up-front investments and continual maintenance. First, you need data. Annotated data can be slow and expensive to acquire manually, especially for tasks that demand critical thinking and domain expertise. Open source data and AI-generated data can mitigate the cost, but their effectiveness is highly variable.

Second, finetuning requires the knowledge of how to train models. You need to evaluate base models to choose one to finetune. Depending on your needs and resources, options might be limited. While finetuning frameworks and APIs can automate many steps in the actual finetuning process, you still need to understand the different training knobs you can tweak, monitor the learning process, and debug when something is wrong. For example, you need to understand how an optimizer works, what

learning rate to use, how much training data is needed, how to address overfitting/underfitting, and how to evaluate your models throughout the process.

Third, once you have a finetuned model, you'll need to figure out how to serve it. Will you host it yourself or use an API service? As discussed in [Chapter 9](#), inference optimization for large models, especially LLMs, isn't trivial. Finetuning requires less of a technical leap if you're already hosting your models in-house and familiar with how to operate models.

More importantly, you need to establish a policy and budget for monitoring, maintaining, and updating your model. As you iterate on your finetuned model, new base models are being developed at a rapid pace. These base models may improve faster than you can enhance your finetuned model. If a new base model outperforms your finetuned model on your specific task, how significant does the performance improvement have to be before you switch to the new base model? What if a new base model doesn't immediately outperform your existing model but has the potential to do so after finetuning—would you experiment with it?

In many cases, switching to a better model would provide only a small incremental improvement, and your task might be given a lower priority than projects with larger returns, like enabling new use cases.^{[2](#)}

AI engineering experiments should start with prompting, following the best practices discussed in [Chapter 6](#). Explore more advanced solutions only if prompting alone proves inadequate. Ensure you have thoroughly tested various prompts, as a model's performance can vary greatly with different prompts.

Many practitioners I've spoken with share a similar story that goes like this. Someone complains that prompting is ineffective and insists on finetuning. Upon investigation, it turns out that prompt experiments were minimal and unsystematic. Instructions were unclear, examples didn't represent actual data, and metrics were poorly defined. After refining the prompt experiment process, the prompt quality improved enough to be sufficient for their application.³